
Méthodes génériques

Pourquoi la généricité

Contexte : écrire une méthode travaillant avec des valeurs d'un type arbitraire/inconnu.

Exemple : une liste n'a pas besoin de connaître le type de ces éléments.

Rappels :

- Les interfaces permettent de généraliser (sous un même type) plusieurs méthodes accomplissant le **même service**, mais avec des **instructions différentes**.
- Une méthode générique généralise plusieurs méthodes ayant les **mêmes instructions**, mais avec des **types différents**.

Exemple

Consulter l'exemple `cours6.printCollection`.

Syntaxe des méthodes génériques

```
public <T> String write(T[] things) {  
    ...  
}
```

- méthode avec des variables de types : *méthode générique*,
- T : *paramètre de généricité*, variable de type,
- déclaration de la variable de type, juste avant le type de retour de la méthode.
- plusieurs variables de type : <S,T,U> ,
- le reste garde la syntaxe usuelle.

Syntaxe des méthodes génériques

```
public <T> String write(T[] things) {  
    ...  
}
```

Cette déclaration induit automatiquement les méthodes suivantes (T abstrait un type référence) :

- `public String write(String[] things);`
- `public String write(Integer[] things);`
- `public String write(People[] things);`
- `public String write(Object[] things);`

mais pas (T n'abstrait pas un type primitif) :

- `public String write(int[] things);`
- `public String write(double[] things);`

Variables de types

- $\langle T \rangle$ déclare une *variable de type*,
- les variables usuelles abstraient des valeurs, on leur substitue la valeur qui leur est donnée *à l'usage*,
- les variables de types abstraient des types, on leur substitue le type adéquat *à l'usage*.
- les types primitifs ne peuvent pas être abstraits.

Interfaces génériques

Les interfaces génériques

Contexte : Deux abstractions simultanément :

- généralisation des **comportements** (blocs d'instruction) par l'interface (même service, instructions différentes),
- généralisation des **types** par la généricité (mêmes instructions, types différents)

Exemples :

- `List<T>`, `Set<T>`, `Map<K, V>` sont des exemples d'interfaces génériques ;
- consulter `cours6.Monoid` et `cours6.reduceList`.

Types génériques, types paramétrés

Type générique

Un *type générique* est un type admettant des paramètres formels de types.

Exemples : `ArrayList<E>`, `Group<T>`, `Iterable<T>`

Type paramétré

Un *type paramétré* est l'*instanciation* d'un type générique avec des paramètres de types concrets.

Exemples : `ArrayList<Integer>`, `Group<String>`, `Iterable<Double>`.

Un exemple : l'interface Comparable

```
public interface Comparable<T> {  
    int compareTo(T value);  
}
```

- compareTo retourne un entier :
 - strictement positif si `this` > value,
 - strictement négatif si `this` < value,
 - nul si `this` = value,
- doit avoir un comportement cohérent avec equals,
- doit avoir un comportement cohérent avec equals,
- utile pour TreeMap, TreeSet, Collections.sort.

Autre exemple : l'interface Comparator<T>

```
public interface Comparator<T> {  
    int compare(T value1, T value2);  
}
```

- compareTo retourne un entier :
 - strictement positif si $value1 > value2$,
 - strictement négatif si $value1 < value2$,
 - nul si $value1 = value2$,
- doit avoir un comportement cohérent avec equals,
- utile pour avoir des ordres autres que celui défini via Comparable.

Classes génériques

Les classes génériques

- une classe paramétrée par un ou plusieurs types ;
- exemple : les classes de collections, paramétrées par le type des éléments ;
- consulter `cours6.IntStack` (non-générique) et `cours6.Stack` (générique).

Classe générique

Déclaration d'une classe générique :

```
public class MyGenericClass<T> { ... }
```

Instanciation d'une classe générique :

```
MyGenericClass<Integer> obj =  
    new MyGenericClass<Integer>(...);  
// or when there is no ambiguity:  
MyGenericClass<Integer> obj =  
    new MyGenericClass<>(...);
```

Tout le reste identique (méthodes, déclaration du constructeur,...).

Limite de la généricité

Une valeur du type paramètre T n'a pas de méthode connue.

Conséquences :

- ces valeurs ne peuvent être utilisées ;
- hormis pour les stocker ;
- cela nous restreint essentiellement aux collections.

Ajout d'un mécanisme de **contrainte de type**. Un paramètre de type T peut avoir des contraintes de la forme :

- T **extends** S (T est un sous-type de I) ;
- T **super** S (S est un sous-type de I).

Consulter cours6.**Maximum**.

Relation de sous-typage

Le **sous-typage** est une relation d'ordre $\preceq$ sur les types.

- Si la classe C implémente l'interface I , $C \preceq I$.
- Si la classe C étend la classe P , $C \preceq P$.
- $\preceq$ est réflexif, anti-symétrique et transitif.

Conséquence sur l'affectation : une valeur v peut être affectée à une variable x si le type de v est un sous-type de celui de x .

Paramètre contraint

Ajouter une contrainte de type `<T extends I>` autorise l'utilisation des méthodes déclarées par I.

Cas de `<T extends Comparable<T>>` :

- T a une méthode `int compareTo(T other)` ;
- possible de calculer un maximum, un tri, ...

Les contraintes de types permettent la conception de classes génériques variées.

Récapitulatif génériques

- Méthodes génériques ;
- interfaces génériques ;
- classes génériques ;
- contraintes d'interface.

```
public <T> Result myMethod(...) { ... }  
public interface myInterface<T> { ... }  
public class myClass<T> { ... }  
  
... <T extends MyInterface> ...
```